

VS CODE PORT FORWARDING GUIDE

A Comprehensive Manual for Exposing Local Servers via Microsoft Dev Tunnels

1. Overview of VS Code Port Forwarding

Port Forwarding is a built-in feature in Visual Studio Code powered by *Microsoft Dev Tunnels*. It enables developers to securely forward local web services or APIs to the Internet via a dedicated HTTPS URL. This eliminates the necessity of utilizing third-party tools such as ngrok or manually reconfiguring router NAT settings.

2. Step-by-Step Implementation Workflow

Step 1: Launch Your Local Application

Verify that your local web application or API server is running properly and accessible on your local machine.

```
# Example: Running a React/Vite development server
npm run dev
# Output in terminal: Local: http://localhost:5173/ or http://127.0.0.1:3000
```

Step 2: Access the Ports Panel in VS Code

You can bring up the Ports management interface through either of the following approaches:

- Navigate via the top menu bar: **View** → **Terminal** → select the **Ports** tab.
- Alternatively, press **Ctrl + Shift + P** (or **Cmd + Shift + P** on macOS), type, and select **Ports: Focus on Ports View**.

Step 3: Specify the Target Port

Click the **Forward a Port** button (or the **+** icon), enter the corresponding port number (e.g., **3000**, **5173**, **8000**, or **8080**), and press **Enter**.

Step 4: Authenticate Account (Initial Setup Only)

During the first execution, VS Code will prompt you to sign in with your **GitHub** or **Microsoft** credentials to initialize a secure tunnel connection.

Step 5: Adjust Port Visibility (Private to Public) & Copy URL

By default, forwarded ports are configured as **Private** (restricted to authenticated account owners only):

- Right-click on the active port entry → choose **Port Visibility** → select **Public**.
- Hover over the **Forwarded Address** column and click the Copy icon (e.g., **https://xxxx-[port].devtunnels.ms**).
- Distribute this secure link to team members, clients, or integrate it with external webhooks and mobile clients for remote testing.

3. Common Practical Use Cases

Application Type	Local Address	Port	Dev Tunnel Public URL
Frontend (React / Vite)	<code>http://localhost:5173</code>	5173	<code>https:// abc123-5173.devtunnels.ms</code>
Fullstack (Next.js / Express)	<code>http://localhost:3000</code>	3000	<code>https:// abc123-3000.devtunnels.ms</code>
Backend API (Django / FastAPI)	<code>http://127.0.0.1:8000</code>	8000	<code>https:// abc123-8000.devtunnels.ms/api/ v1/users</code>
Enterprise API (Spring / .NET)	<code>http://localhost:8080</code>	8080	<code>https:// abc123-8080.devtunnels.ms/ swagger</code>

4. Crucial Security Considerations & Operational Tips

Security Implications of Public Visibility:

- When a port is switched to **Public**, anyone possessing the generated URL can directly interact with your local service.
- **NEVER** forward sensitive database ports (PostgreSQL: 5432, MySQL: 3306, MongoDB: 27017, Redis: 6379), unauthenticated administration panels, or APIs that expose confidential environmental secrets and tokens.

Operational Best Practices:

- **Maintain Active Session:** Both Visual Studio Code and your local development server must remain running. Terminating either process will immediately disconnect the active tunnel.
- **Terminate Port Forwarding:** Once testing is finalized, right-click the port entry and select **Stop Forwarding Port** to securely close the tunnel.
- **Remote Development Environments:** This mechanism operates seamlessly within SSH remote hosts, WSL2, and Docker Dev Containers, automatically routing traffic back to your local client.